DISTRIBUTED ZERO-TRUST ACCESS GATEWAY

Fourth-year capstone project

Installation and User Manual

How to install, configure, operate and remove the gateway

This manual is for whoever has to run the gateway: the student demoing it, the teammate deploying it, and the reviewer checking that it works from a clean machine. It assumes you can use a terminal and that nothing is installed yet.

Every command below has been run against the code in this release. Where a step can fail for a predictable reason, the troubleshooting table in section 9 names the exact error message you will see.

Version 0.1.0 | November 2026 | MIT licence

Source: github.com/mildfriend3-blip/ztag-gateway

Contents

1	What the gateway does
2	Before you begin
3	Install with Docker Compose (recommended)
4	Install from source
5	Your first authorised request
6	Configuration reference
7	Day-to-day operation
8	Deploying to Kubernetes
9	Troubleshooting
10	Security notes
11	Updating and removing
A	Appendix A: token claims
B	Appendix B: Redis keys
C	Appendix C: quick reference

Note. Two installation paths exist. Docker Compose (section 3) brings up Redis, the policy engine, the gateway and two stand-in services with one command and is the path to use for a demo or a review. Building from source (section 4) is for working on the code.

1 What the gateway does

The gateway sits in front of internal services and decides, per request, whether to forward it. It does not check a static API key. For every request it verifies a short-lived signed token, confirms the request comes from the device the token was issued to, checks the token has not been revoked, applies a rate budget, and asks a policy engine whether this identity may reach this route from this service.

Seven stages run in this order, and each one is timed into the audit record:

- **Route match** — longest matching prefix from the configured route table.
- **Token verification** — signature with a pinned algorithm, expiry, issuer, audience, and a ceiling on total lifetime.
- **Device binding** — the fingerprint must be present and must match the device the token was issued to.
- **Revocation** — the token id and the subject are looked up in a local cache, then in Redis.
- **Rate limit** — a sliding window per identity, plus a per-source window on sensitive routes.
- **Authorisation** — scope check, then the policy decision from Open Policy Agent.
- **Forward and audit** — the request reaches the upstream with identity headers; the decision is queued for logging.

A request that fails any stage is rejected before it reaches an internal service, and the rejection reason is recorded. The upstream never receives the bearer token, only the identity the gateway proved.

Note. Authorisation fails **closed**: if the policy engine is unreachable, sensitive routes return 503 rather than being allowed through. Telemetry fails **open**: if the audit sink is down, requests are still served and the events are counted as dropped. That asymmetry is deliberate.

2 Before you begin

Check what you already have. Only one of the two installation paths needs Docker.

Requirement	Version	Needed for
Docker Engine with the Compose plugin	24.0 or newer	Path A. Check with `docker compose version`.
Go toolchain	1.23 or newer	Path B, and any code change. Check with `go version`.
Open Policy Agent binary	1.0 or newer	Path B only. Compose runs OPA in a container.
Redis server	7.0 or newer	Path B only. Compose runs Redis in a container.
A POSIX shell, curl, openssl	any current	Scripts, token generation, smoke checks.
Free disk / memory	~2 GB / ~1 GB	Container images and two services.

The default ports must be free on the machine. Nothing else is required: the gateway binary has no third-party dependencies and no configuration file.

Port	Process	Exposure
8443	gateway (client traffic, metrics, admin API)	Published. This is the only port a client needs.
6379	Redis	Internal in Compose and Kubernetes; published locally for debugging.
8181	Open Policy Agent	Internal. Never expose it to clients.
9000	stand-in services (orders, auth)	Internal. Replace with your real services.

Clone the repository and change into it:

```
git clone https://github.com/mildfriend3-blip/ztag-gateway.git
cd ztag-gateway
```

3 Install with Docker Compose (recommended)

This path starts Redis, the policy engine, the gateway, two stand-in internal services and the audit log worker. It takes about two minutes on a warm machine.

Step 1. Generate local secrets

The repository ships no credentials. This writes a .env file containing a random signing key and admin token. The file is gitignored and must never be committed.

```
make env
cat .env # inspect, do not share
```

Step 2. Start the topology

The first run builds the image, so it takes longer than later runs.

```
make up
```

Step 3. Confirm everything is healthy

The readiness endpoint checks Redis. Both calls should return 200 and the gateway should log one line per configured route.

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8443/healthz
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8443/readyz
make logs
```

Step 4. Run the end-to-end checks

This exercises the allow path, every rejection path, revocation and rate limiting. It prints one line per check and exits non-zero if anything fails.

```
make smoke
```

Step 5. Watch the audit pipeline

Every decision is written to a Redis stream and drained by the log worker into a JSONL file inside the container.

```
docker compose exec logworker tail -f /var/log/ztag/audit.jsonl
```

Important. If `docker compose up` stops with `missing_secret_run_make_env`, step 1 was skipped: the Compose file refuses to start without a signing key on purpose.

4 Install from source

Use this path to modify the code or to run the gateway without containers. You start each dependency yourself.

Step 1. Build the binaries

```
make build
ls bin/ # ztag-proxy, ztag-logworker, ztag-tokengen, ztag-loadgen, ztag-echo
```

Step 2. Start Redis and the policy engine

Any Redis 7 and any OPA 1.x will do. Both must be reachable before the gateway starts; the gateway tolerates a missing Redis at boot but degrades.

```
redis-server --appendonly yes --port 6379 &
opa run --server --addr=0.0.0.0:8181 policies &
```

Step 3. Export the secrets and the route table

There is no default signing key. Without this variable the gateway exits with `ZTAG_JWT_SECRET is required`.

```
export ZTAG_JWT_SECRET=$(openssl rand -hex 32)
export ZTAG_ADMIN_TOKEN=$(openssl rand -hex 24)
export ZTAG_ROUTES='/api/orders=http://127.0.0.1:9000|orders.read|;/api/login=http://127.0.0.1
:9000|auth.login|sensitive'
```

Step 4. Start a stand-in service, the gateway and the log worker

Run each in its own terminal. The echo service reports the headers the gateway forwarded, which is how you confirm the token was stripped.

```
./bin/ztag-echo -addr=:9000 -service=orders
./bin/ztag-proxy
./bin/ztag-logworker -redis=127.0.0.1:6379 -out=/tmp/audit.jsonl
```

Step 5. Verify

```
curl -s http://127.0.0.1:8443/readyz
make smoke
```

5 Your first authorised request

Every request needs two things: a signed token and a device fingerprint. The tokengen tool stands in for your identity provider; in production the IdP issues these and the gateway only verifies them.

Step 1. Issue a token

The claims matter. `-svc` is the calling service, `-did` is the device the token is bound to, and `-scope` must cover the route you are calling.

```
TOKEN=$(./bin/ztag-tokengen -sub user-1001 -svc web -did d-7712 \
-scope orders.read -amr pwd,otp -ttl 5m)
echo $TOKEN
```

Step 2. Call the gateway

The fingerprint header must carry the same device id the token was issued to.

```
curl -s http://localhost:8443/api/orders \
-H "Authorization: Bearer $TOKEN" \
-H 'X-Ztag-Device: id=d-7712;os=macos;managed=1;posture=compliant'
```

With the stand-in service you get the request echoed back, including the identity headers the gateway added:

```
{"service":"orders","method":"GET","path":"/api/orders",
"headers":{"X-Ztag-Device":"d4d1ed5f6eac5324","X-Ztag-Jti":"5ccd2f82...",
"X-Ztag-Request-Id":"11gfr03-1","X-Ztag-Service":"web",
"X-Ztag-Subject":"user-1001","X-Ztag-Trust":"100"}}
```

Note what is absent: there is no `Authorization` header. The gateway consumed the token and replaced it with the identity it proved, so an internal service has no credential to replay.

What a rejection looks like

Rejections are JSON with a stable, greppable reason code. These are the four you will meet first:

```
401 {"error":"missing_bearer_token"}
401 {"error":"invalid_token:bad_signature"}
401 {"error":"device_binding_mismatch"}
403 {"error":"policy_denied:service_not_allowed"}
```

6 Configuration reference

Configuration is entirely environment-driven; there is no config file. Every value has a default except the signing key, and the process validates its configuration at startup, refusing to boot with an unusable security setting.

Variable	Default	Purpose
ZTAG_JWT_SECRET	none — required	HS256 signing key, 16 bytes or more. The gateway will not start without it.
ZTAG_JWT_ALG	HS256	HS256 or RS256. Any other value is refused.
ZTAG_JWT_PUBLIC_KEY_FILE	none	PEM public key, required when the algorithm is RS256.
ZTAG_JWT_ISSUER	ztag-idp	Required iss claim.
ZTAG_JWT_AUDIENCE	ztag-api	Required aud claim.
ZTAG_MAX_TOKEN_TTL	15m	Reject tokens whose exp minus iat exceeds this.
ZTAG_CLOCK_LEEWAY	5s	Tolerance on exp and nbf.
ZTAG_REQUIRE_DEVICE	true	Reject requests without a device fingerprint.
ZTAG_LISTEN	:8443	Listen address.
ZTAG_ROUTES	/api/=http://127.0.0.1:9000|	prefix=upstream|scope|flags, entries separated by a semicolon. The only flag is sensitive.
ZTAG_REDIS_ADDR	127.0.0.1:6379	Redis endpoint.
ZTAG_REDIS_POOL	64	Connection pool size.
ZTAG_REDIS_TIMEOUT	300ms	Per-command deadline.
ZTAG_REVOCATION_NEG_TTL	2s	How long a not-revoked answer is cached in process. Zero means one Redis call per request.
ZTAG_RATE_LIMIT	60	Admissions per window, per identity and per route.
ZTAG_RATE_WINDOW	1m	Sliding window length.
ZTAG_RATELIMIT_FAIL_OPEN	false	If true, fall back to a weaker per-replica limiter when Redis is down.
ZTAG_OPA_URL	http://127.0.0.1:8181/v1/data/ztag/authz/decision	Policy decision endpoint.
ZTAG_POLICY_TIMEOUT	25ms	Latency budget for the policy call.
ZTAG_POLICY_FAIL_OPEN	false	Keep false: a missing decision must deny.
ZTAG_AUDIT_SINKS	redis	redis, jsonl or none.
ZTAG_AUDIT_STREAM	ztag:audit:stream	Stream key used by the sink and the worker.
ZTAG_AUDIT_QUEUE	8192	Queue depth. Full queue means dropped, counted events.
ZTAG_AUDIT_WORKERS	2	Flush workers.
ZTAG_AUDIT_FILE	none	Output path for the jsonl sink. Dash means stdout.
ZTAG_ADMIN_TOKEN	none	Enables the admin API. Empty disables it entirely.
ZTAG_UPSTREAM_TIMEOUT	5s	Upstream response-header timeout.
ZTAG_SHUTDOWN_TIMEOUT	10s	Graceful drain on SIGTERM.

7 Day-to-day operation

Three things matter in normal running: is it healthy, what is it deciding, and can you kill a session quickly.

Health and metrics

Endpoint	Use
GET /healthz	Liveness. Never touches a dependency, so it stays green during a Redis outage.
GET /readyz	Readiness. Pings Redis; a 503 here removes the pod from the load balancer.
GET /metrics	Prometheus text: requests, decisions by outcome, dependency errors, audit emitted and dropped, latency histogram.

The two counters worth an alert are `ztag_dependency_errors_total` and `ztag_audit_events_total{state="dropped"}`. The first tells you which dependency is failing; the second tells you that audit is shedding load.

Revoking a session

Revocation is the incident-response control. Revoking a token id kills one credential; revoking a subject kills every session that user has.

```
curl -X POST localhost:8443/admin/revoke \
-H "X-Ztag-Admin-Token: $ZTAG_ADMIN_TOKEN" \
-H "Content-Type: application/json" \
-d '{"jti":"5ccd2f82388e6a04198d746d","ttl_seconds":900}'
# or every session for one user
-d '{"sub":"user-1001","ttl_seconds":900}'
```

The revocation takes effect on the replica that served the call immediately, and on the others within the negative-cache window (2 seconds by default).

Audit records

Each record carries the decision, the reason, the identity, a device hash and the timing of every stage, which is what makes a latency regression attributable without a profiler.

```
{"ts":1793729137081,"rid":"11gfr03-1","sub":"user-1001","svc":"web",
"did":"d-7712","method":"GET","path":"/api/orders","route":"/api/orders",
"status":200,"decision":"allow","reason":"admitted","lat_us":3484,
"stages_us":{"auth_us":33,"revocation_us":1,"ratelimit_us":1406,
"policy_us":1222,"upstream_us":712}}
```

8 Deploying to Kubernetes

The manifests in `deploy/k8s` assume a namespace called `ztag`, an ingress controller in `ingress-nginx`, and a container image you have published.

Step 1. Publish the image and point the manifests at it

Build with the repository Dockerfile, push to your registry, then set the image name and tag in the kustomization.

```
make image IMAGE=registry.example.com/ztag-gateway:0.1.0
docker push registry.example.com/ztag-gateway:0.1.0
# edit deploy/k8s/kustomization.yaml -> images:
```

Step 2. Create the secret out of band

Do not apply the placeholder Secret file as-is. Create the real secret instead, and delete 02-secret.yaml from the resource list.

```
kubectl create namespace ztag
kubectl -n ztag create secret generic ztag-gateway-secrets \
--from-literal=ZTAG_JWT_SECRET="$(openssl rand -hex 32)" \
--from-literal=ZTAG_ADMIN_TOKEN="$(openssl rand -hex 24)"
```

Step 3. Apply and verify

```
kubectl apply -k deploy/k8s
kubectl -n ztag rollout status deploy/gateway
kubectl -n ztag get pods,hpa,pdb
```

What the manifests give you: three gateway replicas behind a ClusterIP service, a horizontal autoscaler on CPU and in-flight requests, a disruption budget of two, zone spreading, a non-root read-only container, and default-deny network policy. The only permitted paths are gateway to Redis, policy engine and upstreams, log worker to Redis, and DNS. Nothing else in the namespace can talk to anything.

> **Note.** The gateway's route table and the policy engine's route table are two separate lists and must agree. If you add a route to `ZTAG_ROUTES` but not to the Rego data, requests reach the gateway and are denied with `policy_denied:unknown_route`. That is the intended defence in depth, not a bug.

9 Troubleshooting

Each row is an error you can actually hit, in the words the software uses.

Symptom	Cause	Fix
Gateway exits: `ZTAG_JWT_SECRET is required`	No signing key was supplied. There is no default.	Run `make env`, or export the variable.
Compose stops: `missing_secret_run_make_env`	The .env file is absent or the variable is unset.	Run `make env` in the repository root.
503 `policy_unavailable`	The policy engine is down, or slower than the 25 ms budget.	Check the container is running; on a loaded machine raise ZTAG_POLICY_TIMEOUT.
503 `revocation_unavailable` or `ratelimit_unavailable`	Redis is unreachable and the route is marked sensitive.	Restore Redis. For non-sensitive routes the gateway degrades instead.
429 `rate_limited`	The identity window is exhausted. This is the limiter working.	Wait for Retry-After, or raise ZTAG_RATE_LIMIT for that deployment.
401 `invalid_token:ttl_too_long`	The token lives longer than ZTAG_MAX_TOKEN_TTL.	Issue shorter tokens. Do not raise the ceiling to make it pass.
401 `device_binding_mismatch`	The fingerprint id differs from the did claim in the token.	Send the same device id the token was issued for.
401 `device_fingerprint_required`	ZTAG_REQUIRE_DEVICE is true and no fingerprint header was sent.	Send X-Ztag-Device, or set the variable to false for a test rig.
403 `policy_denied:service_not_allowed`	The token's svc claim is not in that route's allowed_services.	Issue the token for the correct service, or change the policy deliberately.
403 `policy_denied:unknown_route`	The route exists in the gateway but not in the policy data.	Add it to policies/data.json and restart or reload OPA.
404 `no_route`	No configured prefix matches the path.	Check ZTAG_ROUTES; prefixes are matched longest first.
No audit lines in the worker	No traffic, a different stream name, or the sink is jsonl only.	Compare ZTAG_AUDIT_STREAM with the worker's -stream flag.
Upstream sees the request but the body is empty	The client sent a body the proxy never forwarded (method without body).	Verify with the echo service, which prints the method it received.

10 Security notes

The gateway is the control point, so its own credentials matter more than most.

- **Never commit .env or a filled-in Secret.** The repository ships placeholders only, and secret scanning with push protection is enabled on it.
- **Rotate by replacing the key.** Changing ZTAG_JWT_SECRET invalidates every token in circulation at once, which is the fastest way to end a suspected compromise; then reissue tokens from the identity provider.
- **Terminate TLS in front of the gateway.** The data plane speaks plain HTTP on its listen address and expects an L7 load balancer to hold the certificate.
- **Keep the admin token long and separate.** It can revoke any session. An empty value disables the admin API, which is the right setting for replicas that never take admin traffic.
- **Do not expose Redis or the policy engine.** Both are internal dependencies; Compose publishes them only for local debugging.
- **Treat the device fingerprint as an assertion.** It defeats casual replay of a stolen token, not an attacker who has also copied the fingerprint. Bind devices with certificates or WebAuthn for that.

11 Updating and removing

Updating is a rebuild and a restart; configuration lives in the environment, so there is no migration step. Roll out one replica at a time and watch the readiness endpoint.

```
git pull
make test && make policy-test
make up # rebuilds the image and recreates changed containers
# Kubernetes
kubectl -n ztag set image deploy/gateway gateway=registry.example.com/ztag-gateway:0.2.0
kubectl -n ztag rollout status deploy/gateway
```

Removing everything, including the Redis volume and the audit log volume:

```
make down # docker compose down -v
kubectl delete -k deploy/k8s # Kubernetes
rm -rf ztag-gateway # source tree
```

Important. The `-v` flag on `docker compose down` deletes the Redis and audit volumes. Take a copy of the audit file first if you need the records.

Appendix A Token claims

Claim	Meaning
sub	The user or principal. Used for revocation by subject.
iss / aud	Must match the configured issuer and audience.
iat / exp / nbf	Lifetime. exp minus iat must not exceed the configured ceiling.
jti	Token id. Used for single-token revocation and correlation in the audit log.
svc	Calling service identity. This is what stops lateral movement.
did	Device id the token is bound to. Must match the fingerprint header.
scope	Route permissions, checked locally and again by the policy engine.
amr	How the user authenticated. Sensitive routes require a second factor here.

Appendix B Redis keys

Key	Type	Purpose
ztag:revoked:jti:<jti>	string with TTL	One revoked token.
ztag:revoked:sub:<sub>	string with TTL	Every session for one user.
ztag:rl:<route>|<subject>	sorted set	Sliding window per identity.
ztag:rl:src|<ip>|<route>	sorted set	Sliding window per source on sensitive routes.
ztag:audit:stream	stream	Audit queue, trimmed to about 200k entries.

Appendix C Quick reference

Command	Does
make env	Write .env with random secrets.
make build	Build every binary into bin/.
make test / race / bench	Unit tests, race detector, micro-benchmarks.
make policy-test	Run the policy unit tests with OPA.
make check	Format, vet, test and policy tests: what CI runs.
make up / down / logs	Start, stop and tail the local topology.
make token	Print a demo token.
make smoke / attack	End-to-end checks, and the three attack replays.
make image	Build the container image.
make k8s-apply / k8s-delete	Apply or remove the Kubernetes manifests.